

VIT BHOPAL

CSE2006 - Programming in Java (Flipped Course)
Build Your Own Project - Project Report

PROJECT REPORT

Library Management System

Submitted by	Dewansh Kejriwal
Registration Number	25BAI10417
Programme / Branch	B.Tech, Computer Science and Engineering(AI&ML)
Course Code & Title	CSE2006 - Programming in Java
GitHub Repository	https://github.com/dewansh25bai10417/library-management-system-java
Date of Submission	18 / 09 / 2026

1. Introduction

Small libraries, like a department or hostel library, are often managed on paper or in a spreadsheet. This gets messy fast — it's hard to tell who has which book, when it's due, or how much fine is pending, without going through every entry by hand.

This project is a **Library Management System** — a command-line Java application that handles the core work of a small library: adding books, registering members, issuing and returning books, calculating late fines, and tracking overdue books. All data is saved to text files so nothing is lost when the program is closed and reopened.

It was built for the CSE2006 Programming in Java flipped-course project, and it uses core Java concepts covered in the course: abstract classes, inheritance, interfaces, custom exceptions, collections, multithreading, and file I/O.

Technologies Used

Item	Detail
Language	Java 17 (core Java only, no external libraries)
Interface	Command-line menu using <code>Scanner</code>
Storage	Plain text files (<code>data/*.txt</code>), pipe-delimited
Key concepts	Abstraction, inheritance, polymorphism, interfaces, custom exceptions, <code>HashMap/ArrayList</code> , multithreading, <code>java.time</code>

2. Problem Statement

2.1 Problem

Manually keeping track of library books — who has borrowed what, when it's due, and how much fine is owed — gets messy fast once you're doing it on paper or a spreadsheet. A small library needs a simple system that keeps this organized without a full database setup.

2.2 Objectives

- Maintain a catalogue of books with available copy counts
- Register members as Student or Faculty, each with different borrowing rules
- Handle issuing and returning books with proper validation
- Automatically calculate late fines based on due dates
- Track overdue books, including automatically in the background
- Save all data so it survives a restart

2.3 Scope

In scope: single-library, command-line system — cataloguing, member registration, issue/return, overdue tracking, and fine calculation, with file-based storage.

Out of scope: multiple branches, online reservations, a GUI, a real database, and login/authentication.

2.4 Target Users

User	Role
Librarian	Uses the CLI to add books, register members, and process issue/return requests
Student member	Max 3 books, 14-day loan, ₹5/day late fine
Faculty member	Max 5 books, 30-day loan, ₹2/day late fine

3. Requirements

3.1 Functional Requirements

Module	Features
Book Management	Add book, remove book, search by title/author, view all books with available copies
Member Management	Register member as Student or Faculty, view all members with books held
Issue / Return	Issue a book (checks availability and borrow limit), return a book (calculates fine automatically)
Overdue Tracking	Background thread checks for overdue books every 60 seconds; on-demand overdue report
Persistence	Load data on start-up, save data on exit and after each change

3.2 Non-Functional Requirements

Requirement	How it's met
Reliability	Custom checked exceptions handle every error case (book not found, member not found, no copies available, limit exceeded) so the program never crashes on bad input
Usability	Simple numbered menu, clear prompts, and friendly error messages instead of stack traces
Maintainability	Code is split into packages by responsibility (model, service, util, exceptions), so new features can be added without touching unrelated code
Performance	HashMap is used for books and members so lookups by ID are fast
Data durability	All data is written to text files and reloaded automatically on the next run

4. System Architecture

The project is organized in layers, so each part only depends on the part below it. This makes it easy to change one part (say, storage) without breaking the rest.

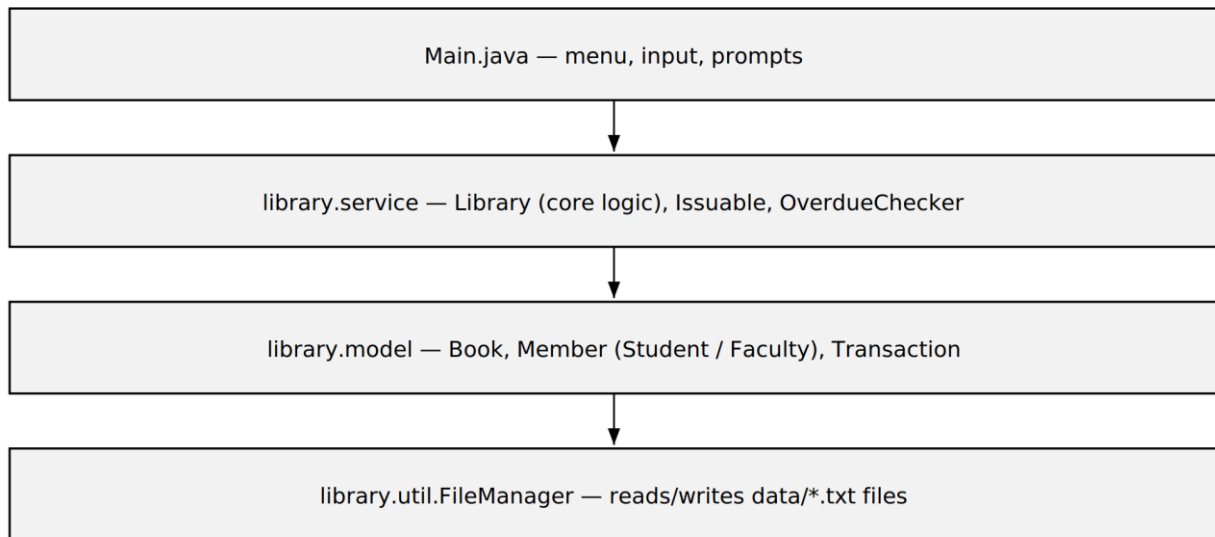


Figure 1 — Layered architecture. The menu calls the service layer, which uses the model classes and saves data through FileManager.

Project structure

```
LibraryManagementSystem/
|-- src/library/
|   |-- Main.java           CLI menu, entry point
|   |-- model/              Book, Member, Student, Faculty, Transaction
|   |-- service/            Library, Issuable, OverdueChecker
|   |-- util/               FileManager
|   +-- exceptions/         custom checked exceptions
|-- data/                   books.txt / members.txt / transactions.txt
+-- README.md
```

5. Class Diagram

`Member` is an abstract class, and `Student` and `Faculty` extend it, each overriding the borrowing limit, loan period, and fine rate. This is the main example of inheritance and polymorphism in the project.

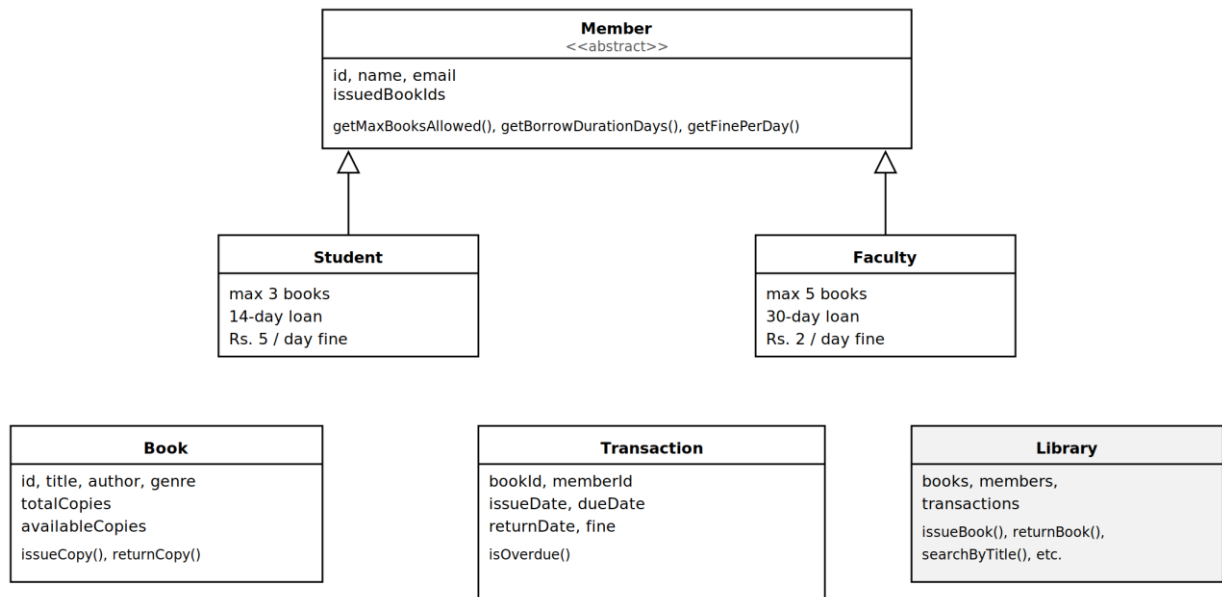


Figure 2 — Class diagram (simplified). Hollow arrows show inheritance from `Member`.

6. Implementation Highlights

A few code snippets that show the main concepts being applied.

6.1 Polymorphism for member rules

Instead of checking "if student then... else if faculty then...", each member type answers for itself:

```
Member member;
if (type.equals("2")) {
    member = new Faculty(id, name, email);
} else {
    member = new Student(id, name, email);
}
library.registerMember(member);
```

Later, `member.getBorrowDurationDays()` and `member.getFinePerDay()` automatically return the right values for that member's type — no extra if-else needed anywhere else.

6.2 Custom exceptions for validation

```
try {
    library.issueBook(bookId, memberId);
    System.out.println("Book issued. Due date is "
        + library.getMember(memberId).getBorrowDurationDays() + " days from today.");
} catch (BookNotFoundException | MemberNotFoundException | BookNotAvailableException
    | BorrowLimitExceededException e) {
    System.out.println("Could not issue book: " + e.getMessage());
}
```

6.3 Background thread for overdue checking

```
OverdueChecker checker = new OverdueChecker(library, 60);
Thread checkerThread = new Thread(checker);
checkerThread.setDaemon(true); // dies automatically when main() ends
checkerThread.start();
```

This runs on its own thread so the overdue check doesn't freeze the menu while it's "sleeping" between checks.

6.4 File storage format

Data is stored using `|` as a separator instead of commas, because book titles and author names often contain commas:

books.txt	bookId title author genre totalCopies availableCopies
members.txt	memberId name email type issuedBookIds
transactions.txt	bookId memberId issueDate dueDate returnDate fine

7. Screenshots / Output

7.1 Program start and menu

```
>> javac -d out (Get-ChildItem -Recurse -Filter *.java -Path src).FullName
>> java -cp out library.Main

=====
Library Management System
=====

--- MENU ---
1. Add Book
2. Remove Book
3. Search Book
4. Register Member
5. Issue Book
6. Return Book
7. View Overdue Books
8. View All Books
9. View All Members
0. Exit
Enter your choice: █
```

7.2 Add a book and register a member

```
Enter your choice: 1
Book ID: B
[Background check] 2 book(s) are currently overdue. Choose option 7 from the menu to view details.
Enter your choice:
Title: Theory of Relativity
Author: Albert Einstein
Genre: Educational
Number of copies: 8
Book added successfully.
```

7.3 Issue a book

```
Book ID: B7
Member ID: M4
Book issued. Due date is 14 days from today.
```

7.4 Return a book

```
>> javac -d out (Get-ChildItem -Recurse -Filter *.java -Path src).FullName
>> java -cp out library.Main
3. Search Book
4. Register Member
5. Issue Book
6. Return Book
7. View Overdue Books
8. View All Books
9. View All Members
0. Exit
Enter your choice:
[Background check] 2 book(s) are currently overdue. Choose option 7 from the menu to view
Enter your choice: 6
Book ID: B7
Member ID: M4
Book returned on time, no fine.
```

7.5 Error handling

```
>> javac -d out (Get-ChildItem -Recurse -Filter *.java -Path src).FullName
>> java -cp out library.Main
1. Add Book
2. Remove Book
3. Search Book
4. Register Member
5. Issue Book
6. Return Book
7. View Overdue Books
8. View All Books
9. View All Members
0. Exit
Enter your choice: 6
Book ID: B13
Member ID: M17
Could not return book: No book found with id B13
```

7.6 Data saved to file

```
LibraryManagementSystem > data > ≡ transactions.txt
1  T1|B1|M1|2026-09-10|2026-09-24|NULL|0.0
2  T2|B3|M2|2026-08-20|2026-09-03|NULL|0.0
3  T3|B5|M4|2026-08-25|2026-09-08|NULL|0.0
4  T4|B7|M1|2026-09-15|2026-09-29|NULL|0.0
5  T5|B4|M5|2026-09-01|2026-10-01|NULL|0.0
6  T6|B2|M3|2026-08-01|2026-08-31|2026-09-02|4.0
7  T7|B7|M4|2026-09-18|2026-10-02|2026-09-18|0.0
8
```

8. Testing Approach

The project was tested manually by running through the main workflow and by deliberately triggering each error case to confirm it's handled gracefully instead of crashing.

Test Case	Expected Result
Add a book, then search for it	Book appears in search results
Issue a book to a member	Available copies decreases by 1
Try to issue a book with an invalid Book ID	"Could not issue book" error, no crash
Issue books until a Student hits their limit of 3	4th issue is rejected with a limit error
Return a book after its due date	Fine is calculated and printed
Exit and restart the program	All books/members/transactions are still there
Enter text instead of a number at a menu prompt	"Please enter a number." and re-prompt, no crash

9. Challenges & Learnings

9.1 Challenges faced

- **Keeping the menu responsive while checking for overdue books** — running the check directly in the menu loop would freeze the program while it "waits". Moving it to a separate daemon thread solved this.
- **Comma-separated data breaking on commas in titles** — switched to `|` as the separator since it doesn't normally appear in book titles or author names.
- **Date arithmetic across month boundaries** — solved by using `java.time.LocalDate` instead of manually adding days.

9.2 Key learnings

- Inheritance is genuinely useful when subclasses have *different behaviour*, not just different data — that's what made the Student/Faculty split work well.
- Custom checked exceptions make error handling much clearer than return codes or generic exceptions.
- Splitting code into packages (model / service / util) made it much easier to change one part without breaking another.

9.3 Future enhancements

- Replace text files with a real database (SQLite/MySQL)
- Add book reservation and renewal
- Add a simple GUI using JavaFX
- Send email/SMS reminders for overdue books

10. References

1. Oracle. *Java Platform, Standard Edition 17 API Specification*.
<https://docs.oracle.com/en/java/javase/17/docs/api/>
2. Oracle. *The Java Tutorials*. <https://docs.oracle.com/javase/tutorial/>
3. CSE2006 Programming in Java — course lecture material and flipped-course modules.
4. VITyarthi. *Build Your Own Project — General Project Instructions and Submission Guidelines*.